

System Architecture Document

Version 1 Offline Dog Detection, First-N Selection, Persistent Tracking, and Video Labelling Application

Document Status: Architecture Specification

Version: 1.0

Application Type: Local Python Desktop Computer-Vision Application

Primary Function: Process prerecorded video, identify the first N distinct dogs, persistently track those dogs, and produce a labelled video available for playback and explicit user saving.

1. Purpose and Scope

This document defines the system architecture for Version 1 of a local Python desktop application that processes prerecorded video files.

The application allows a user to select a local video through a graphical interface. The application then processes the video sequentially, detects dogs using a locally available pretrained Ultralytics YOLO model, identifies the first N distinct dogs that satisfy the configured detection criteria, assigns persistent application-level identities to those dogs, and tracks those selected dogs throughout the remainder of the video.

The processed frames are written to temporary output storage and become available for playback in the GUI when processing completes. The processed video is not permanently saved unless the user explicitly presses the Save button and selects an output destination.

The architecture is intentionally modular. Detector, tracker, video I/O, target selection, rendering, GUI, and application-control responsibilities are separated so that future versions can replace individual implementations without requiring a redesign of the entire application.

V1 remains a local desktop application. It does not introduce networking, databases, cloud infrastructure, distributed processing, microservices, or other infrastructure that is not justified by the current requirements.

2. System Goals

The system shall:

1. Allow the user to select a prerecorded local video.
2. Validate the selected input before processing.
3. Support common video formats including MP4, AVI, MOV, and MKV, subject to installed codec support.
4. Process video sequentially.

5. Detect objects using a locally available pretrained YOLO model.
6. Restrict the V1 detection category to `dog`.
7. Identify the first N distinct dogs that satisfy the configured selection criteria.
8. Assign stable application-level target IDs to selected dogs.
9. Track selected dogs across subsequent frames.
10. Prevent later dogs from replacing already-selected targets merely because they appear later.
11. Handle temporary disappearance and occlusion without immediately abandoning a selected target.
12. Distinguish detector confidence from tracker/association confidence.
13. Draw bounding boxes and labels around selected target dogs.
14. Display labels in the form:

`dog 0.87`

15. Produce a processed video suitable for playback.
 16. Keep the processed video temporary until the user explicitly saves it.
 17. Allow the user to choose the permanent output filename and location.
 18. Provide useful logging and performance measurements.
 19. Fail in bounded, controlled ways.
 20. Clean up resources on successful completion, cancellation, and failure.
 21. Support automated testing without requiring a real YOLO model or GPU for most tests.
 22. Provide clean extension points for future model, tracker, category, and GUI changes.
-

3. Functional Requirements

FR-01: Local video selection

The GUI shall provide a local file-selection dialog.

The application shall not accept network URLs in V1.

FR-02: Input validation

The application shall verify that:

- the selected path exists;
- the path refers to a file;
- the extension is supported;
- the video can be opened;

- at least one valid frame can be read.

Metadata such as FPS and frame count shall be treated as potentially unreliable rather than assumed to be correct.

FR-03: Dog detection

The detector shall use a locally available pretrained YOLO model capable of detecting the COCO dog class.

The application shall filter detections so that only the configured V1 target category is passed into target selection and tracking.

FR-04: First-N selection

The system shall select at most N distinct dogs.

N shall be defined in configuration and shall not be entered by the user through the GUI.

For example:

```
TARGET_CLASS = "dog"  
MAX_TARGETS = 3
```

If three qualifying dogs are selected, a fourth dog appearing later shall not replace any of them.

FR-05: Persistent tracking

Each selected dog shall have an application-level persistent target ID.

For example:

```
Target 1  
Target 2  
Target 3
```

These identifiers are independent of raw YOLO result objects and should not depend on the numerical IDs produced internally by a particular tracker.

FR-06: Annotation

Selected target dogs shall be annotated with:

- bounding box;
- class name;
- current detector confidence when a detector observation is available.

Example:

```
dog 0.87
```

FR-07: Temporary output

Processed frames shall be written to a temporary video file.

The temporary result shall remain available after processing so that the GUI can play it.

FR-08: Explicit permanent save

The application shall not automatically create a permanent user output.

The user must explicitly invoke Save and select a destination.

FR-09: Playback

After successful processing, the GUI shall allow playback of the processed video.

FR-10: Offline execution

Runtime processing shall not require:

- Internet;
- cloud services;
- remote APIs;
- network connectivity;
- model downloads;
- telemetry.

Model weights shall be provisioned during installation or device setup.

4. Non-Functional Requirements / Quality Attributes

4.1 Maintainability

Major computer-vision responsibilities shall be isolated behind application-level interfaces.

The GUI shall not directly manipulate YOLO result objects.

The detector shall not be responsible for target-selection policy.

The tracker shall not determine whether an object qualifies as one of the first N targets.

4.2 Extensibility

The architecture shall permit:

- another YOLO model;
- another detector;
- another tracker;
- additional object categories;
- different annotation policies;

- additional output formats;
- alternative video readers/writers.

without rewriting the complete application.

4.3 Reliability

The system shall use bounded retry behavior and bounded error logging.

No component shall contain an infinite retry loop.

4.4 Performance

Performance shall be measured rather than promised.

The application shall record actual:

- processing FPS;
- inference latency;
- total processing time;
- CPU usage;
- memory usage;
- GPU usage where available.

Final acceptable throughput shall be established by benchmark testing against representative hardware and video.

4.5 Usability

The GUI shall provide clear state transitions and meaningful error messages.

Technical exception traces shall not be shown directly to normal users.

4.6 Portability

The application should use cross-platform Python libraries and avoid OS-specific video or GUI APIs wherever practical.

4.7 Data minimization

The application shall not maintain a database or persistent detection history.

Temporary files shall be deleted when they are no longer required.

5. V1 Scope and Explicit Out-of-Scope Items

V1 includes:

- local prerecorded video;
- DOG detection;
- first-N target selection;
- persistent target tracking;
- bounding-box annotation;
- temporary processed-video generation;
- GUI playback;
- explicit user-controlled saving;
- offline operation;
- performance logging;
- error handling;
- automated testing.

The following are explicitly out of scope:

- live camera capture;
- webcam access;
- network cameras;
- RTSP;
- cloud inference;
- remote APIs;
- internet processing;
- cloud storage;
- databases;
- detection history databases;
- user accounts;
- authentication;
- multi-user operation;
- web applications;
- notifications;
- automatic uploads;

- custom model training;
- training infrastructure;
- runtime model downloads;
- standalone executable packaging;
- microservices;
- message brokers;
- distributed processing;
- automatic permanent output storage;
- batch-processing orchestration;
- complex persistent processing pipelines.

The architecture may contain interfaces that make these future capabilities possible, but V1 shall not implement them.

6. Architectural Principles

6.1 Separation of concerns

Detection, target selection, tracking, rendering, and GUI control are separate responsibilities.

6.2 Application-owned data contracts

YOLO-specific result objects shall be converted into application-level data structures at the detector boundary.

6.3 Framework isolation

Ultralytics APIs shall not propagate throughout the application.

6.4 Target identity is application state

A tracker ID is not treated as the application's permanent identity.

The application maintains its own target identity and maps tracker observations to that identity.

6.5 Sequential processing

Frames are processed in source order.

Tracking state depends on frame continuity.

6.6 Bounded failure handling

Failures are retried only where retry is meaningful.

Repeated failures are aggregated or rate-limited.

6.7 Temporary-first output

Processing output is temporary until explicitly saved.

6.8 Testability

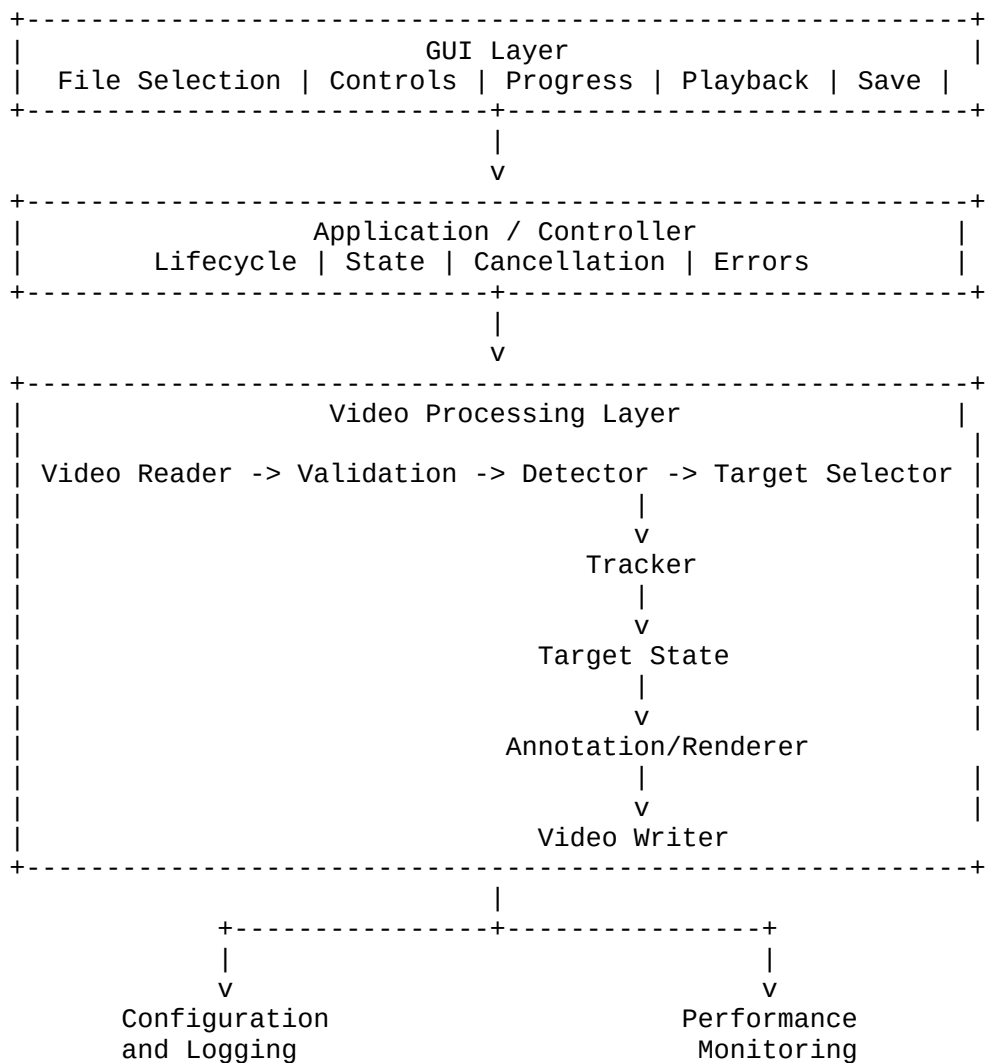
Major external dependencies are represented through lightweight interfaces that can be mocked.

6.9 Minimal V1 architecture

Abstractions are introduced where they protect a genuine architectural boundary, not merely to create additional classes.

7. High-Level Architecture

The system is divided into six logical layers.



The GUI communicates with the application controller.

The controller owns the processing session.

The processing pipeline owns the frame-by-frame computer-vision workflow.

The detector produces application-level detections.

The target selector determines which dogs become permanent targets.

The tracker maintains frame-to-frame associations.

The target state manager translates tracker observations into application-level target state.

The renderer produces labelled frames.

The video writer produces the temporary output.

8. Component Architecture

The major components are:

1. GUI
2. Application Controller
3. Video Reader
4. Frame Validator
5. Detector
6. Detection Adapter
7. Target Selector
8. Tracker
9. Target State Manager
10. Renderer
11. Temporary Video Writer
12. Playback Controller
13. Save Manager
14. Configuration
15. Logging
16. Performance Monitor

The architecture does not require every component to be a large independent class. Some can remain small modules.

9. Component Responsibilities

9.1 GUI

Responsible for:

- file selection;
- displaying selected filename;
- starting processing;
- displaying progress;
- displaying processing status;
- displaying errors;
- playback;
- Save operation;
- cancellation.

The GUI does not:

- run YOLO;
- interpret raw tracker results;
- decide target identity;
- write processed frames;
- manage model loading.

9.2 Application Controller

The controller coordinates the application lifecycle.

Responsibilities include:

- validating configuration;
- creating dependencies;
- opening input;
- initializing detector/tracker;
- starting processing;
- handling cancellation;
- reporting progress;
- handling errors;
- closing resources.

It is the primary orchestration boundary.

9.3 Video Reader

Responsible for:

- opening local video;
- retrieving metadata;
- reading frames;
- detecting EOF;
- exposing source properties;
- releasing the capture resource.

OpenCV `VideoCapture` is appropriate for V1.

9.4 Frame Validator

Checks:

- frame is not `None`;
- frame has expected dimensionality;
- frame contains usable pixel data;
- frame dimensions are valid.

Invalid frames shall not be passed to inference.

9.5 Detector

Provides:

```
detect(frame) -> list[Detection]
```

The detector is responsible only for model inference and conversion of results into the application representation.

9.6 Detection Adapter

Converts framework-specific YOLO results into:

`Detection`

This prevents Ultralytics result structures from becoming application-wide dependencies.

9.7 Target Selector

Responsible for determining when a detected dog qualifies as one of the first N targets.

It maintains selection state but does not perform general tracking.

9.8 Tracker

Responsible for temporal association.

Conceptually:

```
update(detections, frame) -> list[TrackObservation]
```

The tracker determines which current observations correspond to existing tracks.

9.9 Target State Manager

This is an important additional boundary.

It maps:

```
tracker track -> application target
```

and maintains:

- application target ID;
- current tracker ID;
- lifecycle state;
- last known bounding box;
- last detection confidence;
- last-seen frame;
- visibility state;
- association confidence.

This prevents the application from assuming that a tracker-generated ID is itself the permanent business identity.

9.10 Renderer

Responsible for:

- bounding boxes;
- labels;
- optional target ID;
- visual styling;
- rendering only selected targets.

It receives application-level target state rather than raw YOLO results.

9.11 Temporary Video Writer

Responsible for encoding processed frames into temporary output.

9.12 Playback Controller

Loads the completed temporary result into the GUI playback mechanism.

9.13 Save Manager

Copies or moves the temporary processed result to the user-selected destination after explicit Save.

The Save Manager must validate:

- destination directory;
- filename;
- overwrite policy;
- available disk space;
- output format/codec.

9.14 Configuration

Owns all V1 operational parameters.

9.15 Logging

Provides application-wide logging.

9.16 Performance Monitor

Measures processing performance independently from application flow logging.

10. Interfaces and Data Contracts

The application should use simple Python protocols or abstract base classes where substitution provides clear testing or extensibility value.

10.1 Detection

Conceptual structure:

```
Detection
  class_id
  class_name
  confidence
  x1
  y1
  x2
  y2
```

Coordinates should use a documented convention, preferably pixel coordinates relative to the current frame.

10.2 TrackObservation

```
TrackObservation
    tracker_id
    detection
    association_confidence
    state
```

association_confidence is logically distinct from detector confidence.

A tracker may determine that an observation is likely to correspond to a previous track even when the detector confidence changes.

10.3 TargetObject

```
TargetObject
    target_id
    class_name
    tracker_id
    latest_detection
    last_bbox
    last_seen_frame
    lifecycle_state
    visibility_state
    association_confidence
```

Possible lifecycle states:

```
CANDIDATE
ACTIVE
TEMPORARILY_LOST
REACQUIRED
PERMANENTLY_LOST
```

A simpler implementation can collapse some states, but these concepts should remain explicit.

10.4 VideoMetadata

```
VideoMetadata
    width
    height
    source_fps
    frame_count
    duration
    frame_count_known
    fps_reliable
```

Metadata values must be treated as potentially unknown.

10.5 Detector Interface

```
Detector
    load()
    detect(frame)
    close()
```

The actual implementation wraps Ultralytics.

10.6 Tracker Interface

```
Tracker
    initialize()
    update(frame, detections)
    reset()
    close()
```

The interface deliberately does not expose `model.track()` or a specific tracker YAML format.

10.7 Video Reader Interface

```
VideoReader
    open(path)
    metadata()
    read()
    is_open()
    close()
```

10.8 Video Writer Interface

```
VideoWriter
    open(path, metadata)
    write(frame)
    close()
```

10.9 Renderer Interface

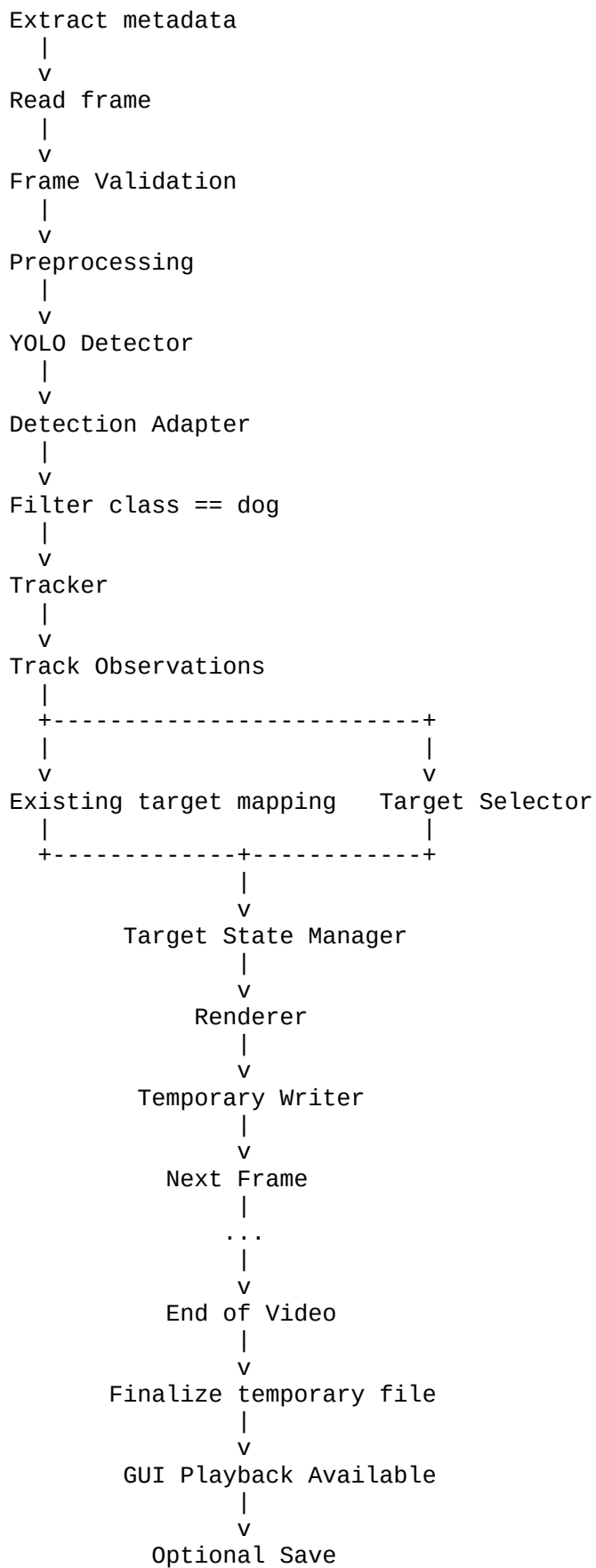
```
Renderer
    render(frame, targets)
```

These interfaces should remain lightweight.

11. End-to-End Data Flow

The complete flow is:

```
User
|
v
GUI
|
v
Select local video
|
v
Application Controller
|
v
Validate path
|
v
Video Reader
|
v
```



The target-selection step and tracking step are related but separate.

Tracking answers:

Which current detection corresponds to which previous track?

Target selection answers:

Is this physical dog one of the first N dogs that the application has selected?

12. Video Input Architecture

OpenCV `VideoCapture` is appropriate for V1 because it provides local video decoding through a widely used API and avoids introducing another dedicated video-processing framework.

The reader shall:

1. Verify the path.
2. Attempt to open the file.
3. Confirm that the capture object reports an open stream.
4. Read an initial frame.
5. Validate that frame.
6. Retrieve metadata.
7. Begin sequential processing.

Supported extensions:

.mp4
.avi
.mov
.mkv

Extension support does not guarantee codec support.

Therefore:

supported extension
 !=
guaranteed decodable video

The application shall report an appropriate error when the container or codec cannot be decoded.

Frame count

Frame count shall be treated as optional metadata.

It may be used for progress if reliable.

If unavailable or obviously invalid, the GUI shall show indeterminate progress or a status message rather than fabricate a percentage.

FPS

Source FPS is informational and used for output timing and comparison.

It shall not be confused with processing FPS.

Corrupt frames

A single malformed frame may be handled according to configured policy.

A practical V1 policy is:

- tolerate a small number of isolated invalid reads;
- log warnings;
- continue where OpenCV can recover;
- terminate after a bounded consecutive failure threshold.

The application must not continue indefinitely while receiving invalid frames.

13. YOLO Detection Architecture

Ultralytics is the primary implementation candidate.

Current Ultralytics documentation lists YOLOv2 and YOLOv11 among its supported model families and provides Python APIs for prediction and tracking. The implementation team should select the appropriate stable model available at implementation time rather than hard-coding an obsolete model choice in this architecture document. ([Ultralytics Docs](#))

The selected model must:

- support object detection;
- use a COCO-compatible pretrained model;
- include the `dog` category;
- be available locally;
- be compatible with the selected installed Ultralytics version;
- be benchmarked on the intended hardware.

Model-selection tradeoff

A smaller model generally offers:

- lower memory consumption;
- lower inference latency;
- better CPU suitability;
- easier deployment on lower-end systems.

A larger model generally provides:

- better detection accuracy;

- improved small-object recognition in many circumstances;
- higher computational cost;
- greater memory requirements.

For V1, a small model should be the initial candidate because the workload is video processing on potentially CPU-only machines.

However, model selection shall be based on benchmark results rather than the assumption that the smallest model is always sufficient.

The selected model and exact package version shall be recorded in project documentation.

Device configuration

Example:

```
DEVICE = "auto"
```

Possible values:

```
auto  
cpu  
cuda
```

If `cuda` is explicitly requested and unavailable, startup shall fail clearly.

`auto` may select the best supported local device.

The application shall not silently change from explicitly requested CUDA to CPU.

Model loading

Model loading occurs before video processing.

Failure to load the model is a startup failure, not a per-frame recoverable error.

14. First-N Target Selection Architecture

This is distinct from detection and tracking.

The system needs to answer a specific question:

Which physical dogs are the first N qualifying dogs encountered in temporal video order?

14.1 Definition of "first"

The first dog is the earliest distinct physical dog that:

1. is detected as class `dog`;
2. meets the configured detector confidence threshold;

3. participates in a valid tracker association;
4. remains sufficiently stable for the configured confirmation criteria.

If several dogs first become qualifying targets in the same frame, their ordering shall be deterministic.

A practical rule is:

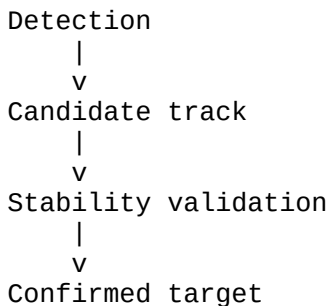
1. sort qualifying candidate tracks by bounding-box horizontal position from left to right;
2. use tracker ID as a secondary deterministic tie-breaker.

This means simultaneous detections do not depend on arbitrary dictionary or framework ordering.

14.2 Candidate versus target

A detector observation shall not immediately become a permanent target.

The pipeline should use:



A practical V1 confirmation policy is based on consecutive observations.

For example:

```
TARGET_CONFIRMATION_FRAMES = 3
```

A candidate must be associated with the same tracker identity for the required number of consecutive frames before it consumes one of the N target slots.

This value is configurable.

The exact default should be validated against the model, video resolution, and frame rate.

14.3 Why confirmation is required

Immediate assignment causes transient detections to consume target slots.

Example:

Frame 100:
dog detection appears for one frame

Frame 101:
detection disappears

Frame 102:
actual dog appears

Without confirmation, the transient false positive could permanently consume one of the N slots.

14.4 Duplicate prevention

The same physical dog must not consume multiple target slots.

The system should first rely on tracker continuity.

A candidate associated with an existing target's active tracker ID cannot create a new target.

If the tracker produces a new ID after temporary loss, the target state manager must attempt target re-association before permitting the observation to become a new target.

A new target may only be created if:

```
confirmed candidate
AND
candidate does not map to an existing target
AND
target_count < N
```

14.5 Long disappearance

A tracker cannot guarantee physical identity indefinitely.

Therefore V1 shall distinguish:

temporary tracking loss

from:

identity no longer confidently maintainable

During the tracker buffer period, the target remains reserved.

If the dog disappears temporarily and the tracker retains the track, the application continues to regard it as the same target.

If the dog returns with the same tracker identity, the target becomes active again.

If a completely new tracker ID is created after the original track is removed, target-level re-association is required before considering the object the same dog.

BoT-SORT with ReID can improve this case, but no visual tracker should be represented as providing absolute identity guarantees.

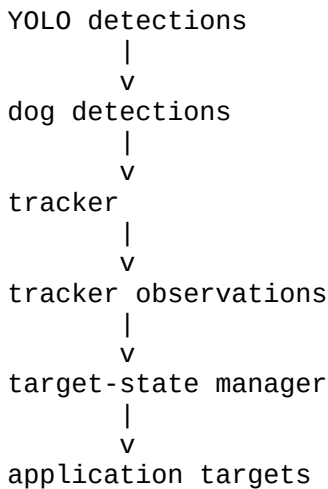
If identity cannot be confidently re-established, the system shall mark the original target as **PERMANENTLY_LOST** rather than silently assigning another dog to its identity.

This is important because replacing the original dog with a later dog would violate the first-N requirement.

15. Object Tracking Architecture

Tracking is an explicit pipeline stage.

The architecture is:



YOLO detection alone does not provide persistent identity.

The tracker receives temporally consecutive frames and maintains state across them.

Ultralytics currently exposes tracker integrations including ByteTrack and BoT-SORT and supports persistent tracking across consecutive frames through its tracking API. The application should wrap this framework functionality rather than allowing Ultralytics-specific tracker objects to become application-wide contracts. ([Ultralytics Docs](#))

16. ByteTrack vs. BoT-SORT Evaluation

16.1 ByteTrack

ByteTrack is fundamentally an association-based multi-object tracker that uses motion prediction and bounding-box association. A significant practical characteristic is its use of lower-confidence detections for recovering existing tracks rather than simply discarding them.

Ultralytics describes ByteTrack as using a linear Kalman motion model, no appearance/ReID component, and a two-stage low-confidence association strategy. ([Ultralytics Docs](#))

Advantages

- relatively low computational overhead;
- straightforward tracking architecture;
- good real-time performance;
- no additional appearance model;
- suitable for CPU-oriented systems;

- simpler configuration;
- useful when objects are visually distinguishable primarily through position and motion.

Disadvantages

- no appearance-based ReID;
- more vulnerable to ID switches when similar objects overlap;
- limited ability to recognize a dog after a long disappearance;
- no camera-motion compensation;
- two visually similar dogs crossing can cause identity ambiguity.

Occlusion

Short occlusion can be handled using tracker state and motion prediction.

Long occlusion is more difficult because the tracker has less information about the object's identity.

Multiple similar dogs

This is a significant weakness.

Two similar dogs crossing paths can cause track association errors.

CPU/GPU

ByteTrack has relatively low overhead compared with trackers using appearance embeddings.

It is therefore attractive for CPU-only V1 deployments.

16.2 BoT-SORT

BoT-SORT combines motion-based tracking with additional association mechanisms. Ultralytics documents BoT-SORT as using a linear Kalman model, optional ReID, and configurable global-motion compensation. ([Ultralytics Docs](#))

Advantages

- stronger identity preservation in difficult scenes;
- optional appearance/ReID information;
- camera-motion compensation;
- better suitability for moving-camera footage;
- stronger potential for re-identification after occlusion;
- more appropriate for scenes containing several visually similar dogs.

Disadvantages

- more computational overhead;
- more configuration complexity;
- ReID can increase CPU/GPU and memory requirements;
- additional parameters make tuning more involved;
- unnecessary overhead for very simple static scenes.

Occlusion

BoT-SORT's track buffer and optional appearance matching provide more information for recovering an object after partial occlusion.

Multiple similar dogs

ReID can provide an appearance cue in addition to spatial proximity.

This makes BoT-SORT more appropriate when two dogs are visually or spatially similar.

CPU/GPU

BoT-SORT without ReID is relatively practical on CPU.

BoT-SORT with ReID adds computational cost and should be benchmarked.

16.3 Comparison

Characteristic	ByteTrack	BoT-SORT
Motion tracking	Yes	Yes
Low-confidence association	Yes	Yes
ReID	No	Optional
Camera-motion compensation	No	Yes
CPU cost	Lower	Higher
Configuration complexity	Lower	Higher
Similar-object separation	Moderate	Better with ReID
Long occlusion recovery	Limited	Better
Moving camera	Weaker	Stronger
Static prerecorded video	Very suitable	Suitable
V1 implementation complexity	Lower	Moderate
Identity robustness	Moderate	Higher potential

17. Tracker Selection and Abstraction

17.1 V1 recommendation

BoT-SORT is recommended as the primary V1 tracker, with ReID treated as a configurable capability rather than an unconditional requirement.

The reason is the application's unusual requirement:

The first N physical dogs must retain their identities for the remainder of the video, and later dogs must not replace them.

This requirement places greater importance on identity stability than a conventional "draw tracks around currently visible objects" application.

BoT-SORT provides optional appearance-based association and camera-motion compensation, giving the application additional mechanisms for maintaining identity. ([Ultralytics Docs](#))

However, the architecture shall not depend specifically on BoT-SORT.

17.2 Initial V1 configuration

A practical implementation should initially benchmark:

BoT-SORT
ReID = disabled

against:

BoT-SORT
ReID = enabled

If identity switches or long-occlusion failures are unacceptable without ReID, ReID can be enabled.

This avoids forcing the additional cost on every machine without evidence that it is necessary.

ByteTrack shall remain a supported alternative through the tracker abstraction.

17.3 Tracker abstraction

The application shall define its own tracker interface.

The implementation may internally use:

Ultralytics + BoT-SORT

or:

Ultralytics + ByteTrack

but the rest of the application sees only:

Tracker

The GUI, renderer, target selector, and configuration model should not contain code such as:

```
if tracker == "botsort":
```

except inside tracker-specific configuration/adaptor code.

This allows future tracker replacement without changing the processing pipeline.

18. Video Processing Pipeline

Each frame follows this sequence:

Step 1: Read

Read the next sequential frame.

Step 2: Validate

Reject null or malformed frames.

Step 3: Preprocess

Only perform preprocessing that is justified by benchmark results.

Potential operations include:

- resizing;
- color conversion;
- normalization.

The detector should receive the representation expected by the selected YOLO API.

Unnecessary duplicate conversion should be avoided.

Step 4: Detect

Run YOLO inference.

Step 5: Convert

Convert YOLO results into application-level `Detection` objects.

Step 6: Filter

Keep only:

```
class_name == dog
```

and detections meeting the configured detector threshold.

Step 7: Track

Update the selected tracker using the current frame and dog detections.

Step 8: Associate

Map tracker observations to existing application targets.

Step 9: Select

If fewer than N targets exist, evaluate new confirmed candidate tracks.

Step 10: Update state

Update:

- target position;
- tracker ID;
- last seen frame;
- visibility;
- confidence;
- lifecycle state.

Step 11: Render

Draw only selected targets.

Step 12: Write

Write the annotated frame to temporary output.

Step 13: Metrics

Update performance measurements.

Step 14: Progress

Update GUI progress if reliable frame-count information exists.

19. Annotation and Rendering

The renderer shall consume application-level target state.

For an active target:

```
+-----+
| dog 0.87 |
|         |
| bounding box |
|         |
+-----+
```

The renderer should not call YOLO methods.

The renderer may optionally display an internal target ID during debug mode, for example:

Target 2 | dog 0.87

However, V1 normal display should remain focused on the requested:

dog 0.87

When a target is temporarily lost, the renderer shall not draw a fabricated current bounding box.

A configurable debug mode may instead show:

dog 0.87 [LOST]

or a last-known marker, but this is not required for normal V1 presentation.

20. Temporary Output and Explicit Save Architecture

This is a key V1 boundary.

20.1 Processing output

The processing pipeline writes to an application-managed temporary file.

Conceptually:

```
temporary_directory/  
  processing_session_xxxxx/  
    processed_video.mp4
```

The temporary directory should use a collision-resistant session-specific name.

20.2 Why temporary storage

The processed video must exist after processing because playback needs a complete file.

At the same time, the application must not permanently save the user's processed video without explicit consent.

Temporary output satisfies both requirements.

20.3 Playback

When processing completes:

```
temporary output  
  |  
  v  
Playback Controller  
  |  
  v  
GUI video player
```

The temporary file remains available while the application session remains active.

20.4 Save

When the user presses Save:

1. open destination picker;
2. obtain destination path;
3. validate destination;
4. verify sufficient disk space where practical;
5. copy the temporary output to the selected destination;
6. verify the operation;
7. report success.

The original temporary output remains the playback source.

20.5 Temporary cleanup

Temporary output shall be removed:

- after explicit application shutdown;
- after a failed processing session;
- when replaced by a newer processing session;
- when no longer required.

Cleanup should use best-effort deletion with errors logged.

A failure to delete a temporary file should not cause an otherwise successful processing operation to be reported as failed.

21. GUI Architecture

21.1 GUI framework selection

PySide6 is recommended for V1.

Qt for Python provides official Python bindings for Qt 6 through PySide6, including Qt Widgets and Qt Core functionality. ([Qt Documentation](#))

PySide6 is preferred over Tkinter because this application requires:

- video playback;
- progress controls;
- richer widget support;
- asynchronous worker integration;

- scalable desktop UI;
- more sophisticated future GUI capabilities.

Tkinter remains viable for a much simpler interface, but PySide6 provides a stronger foundation for this application's expected evolution without requiring a web stack.

Qt's signal/slot mechanism also provides a clean way to communicate between a worker and the GUI thread.

21.2 GUI states

The GUI should have explicit states:

IDLE
FILE_SELECTED
VALIDATING
PROCESSING
CANCELLING
FAILED
COMPLETED
PLAYBACK
SAVING
SAVE_FAILED

IDLE

Enabled:

- Select Video

Disabled:

- Process
- Save
- Playback

FILE_SELECTED

Enabled:

- Process

VALIDATING

Display:

Validating video...

Processing controls should be disabled.

PROCESSING

Display:

- current status;

- progress if reliable;
- elapsed time;
- optional processing FPS.

The GUI remains responsive.

FAILED

Display a user-readable error.

Example:

Video processing failed.
The selected video could not be decoded.
See the application log for diagnostic information.

COMPLETED

Enable:

- playback;
- Save.

PLAYBACK

Allow the user to play, pause, seek where supported, and stop playback.

SAVING

Disable duplicate Save requests.

SAVE_FAILED

Retain the temporary result so that the user can retry saving.

22. Configuration Architecture

A centralized configuration module shall be used.

Example categories:

```
# Target
TARGET_CLASS = "dog"
MAX_TARGETS = 3
DETECTION_CONFIDENCE = 0.40
TARGET_CONFIRMATION_FRAMES = 3

# Model
MODEL_PATH = "models/yolo_model.pt"
DEVICE = "auto"

# Tracker
TRACKER_TYPE = "botsort"
TRACK_HIGH_THRESHOLD = ...
```

```

TRACK_LOW_THRESHOLD = ...
NEW_TRACK_THRESHOLD = ...
TRACK_BUFFER = ...
MATCH_THRESHOLD = ...
REID_ENABLED = False

# Video
SUPPORTED_EXTENSIONS = (...)
OUTPUT_CODEC = ...
OUTPUT_EXTENSION = ...

# Processing
PROCESSING_SIZE = ...
PERFORMANCE_LOG_INTERVAL = 5
MAX_CONSECUTIVE_READ_FAILURES = ...
MAX_CONSECUTIVE_INFERENCE_FAILURES = ...

# Logging
LOG_LEVEL = "INFO"

# GUI
WINDOW_TITLE = ...

```

Exact tracker parameters shall correspond to the selected tracker and installed Ultralytics version.

Confidence threshold

A starting value around 0.40 may be reasonable for initial validation, but it is not an industry-standard universal threshold.

The correct value depends on:

- model family;
- model size;
- video resolution;
- lighting;
- camera quality;
- required recall;
- false-positive tolerance.

Therefore the threshold must remain configurable and be tuned through validation.

23. Logging and Debugging Strategy

Console logging is sufficient for V1.

Recommended levels:

DEBUG

Detailed diagnostic information:

- frame-level tracker events;
- target associations;
- state changes;
- retry counts;
- performance samples.

INFO

Normal lifecycle:

- startup;
- configuration;
- model loaded;
- input selected;
- metadata;
- processing start;
- target selected;
- completion;
- save.

WARNING

Recoverable problems:

- invalid frame;
- isolated read failure;
- temporary target loss;
- unreliable metadata;
- GPU metric unavailable.

ERROR

Operation failures:

- model load failure;
- unrecoverable video error;
- writer failure;
- inference failure threshold exceeded;
- save failure.

TRACE

Not required for normal V1.

If implemented, it should be reserved for very high-volume debugging and must not be enabled by default.

Repeated failure handling

The application shall not log:

```
ERROR frame read failed
ERROR frame read failed
ERROR frame read failed
...
```

indefinitely.

Instead:

- log the first failure;
- aggregate subsequent failures;
- periodically report count;
- terminate after the configured maximum.

Example:

```
WARNING: frame read failures detected
WARNING: 50 consecutive frame-read failures
ERROR: maximum frame-read failure threshold exceeded
```

24. Performance Monitoring and Metrics

Performance measurement must distinguish several different rates.

24.1 Source FPS

The video's nominal FPS.

Example:

Source FPS = 30

This does not mean the application processes 30 FPS.

24.2 Processing FPS

Actual application throughput:

$\text{processed_frames} / \text{processing_elapsed_seconds}$

This should exclude unrelated GUI idle time.

For example:

Video source FPS: 30.0
Processing FPS: 18.7

means the application is processing slower than real-time.

24.3 Inference time

Measured around the detector invocation:

```
inference_start
  |
  v
detector.detect()
  |
  v
inference_end
```

Inference latency should be aggregated rather than logging every frame.

Metrics should include at least:

- mean inference time;
- optionally median;
- optionally percentile such as p95.

24.4 Total processing time

Measured from the beginning of actual frame processing until the final frame is written.

It should not include unrelated application startup time unless explicitly reported as a separate metric.

24.5 CPU utilization

A system monitoring library such as `psutil` may be used.

CPU measurements should be sampled periodically rather than collected unnecessarily every frame.

24.6 Memory utilization

Memory usage should be sampled periodically.

24.7 GPU utilization

GPU utilization is optional because it may not be available on all platforms.

When unavailable:

GPU utilization: unavailable

is preferable to an invented value.

24.8 Sampling

Performance measurements should use an aggregation window.

For example:

PERFORMANCE_LOG_INTERVAL = 5 seconds

Every interval, report:

Processing FPS
Average inference time
CPU utilization
Memory utilization
GPU utilization if available
Frames processed

At completion, report the final aggregate:

Total frames
Total processing time
Average processing FPS
Average inference time
Peak/average memory
CPU summary
GPU summary

24.9 Warm-up

The first few inference operations may be slower because of:

- model initialization;
- runtime initialization;
- GPU initialization;
- memory allocation.

The implementation should either:

- exclude a small configured warm-up interval from steady-state inference statistics; or
- explicitly report startup and steady-state metrics separately.

No arbitrary performance guarantee should be placed in the SAD.

25. Error and Exception Handling

25.1 Nonexistent input

Detection:

```
Path.exists() == False
```

Behavior:

- log ERROR;
- display user-facing validation message;
- remain in file-selection state.

25.2 Invalid input path

Same as nonexistent input.

No processing starts.

25.3 Unsupported extension

Detection:

```
suffix not in SUPPORTED_EXTENSIONS
```

Behavior:

- WARNING or ERROR depending on context;
- reject input.

25.4 Unsupported codec/container

Detected when OpenCV cannot open/read the video.

Behavior:

- ERROR;
- user-readable message;
- no processing;
- clean up capture.

25.5 Zero-frame video

If the reader reaches EOF without a valid frame:

- ERROR;
- processing fails;
- temporary output is deleted.

25.6 Missing metadata

If FPS or frame count is missing:

- log WARNING;

- continue where possible;
- progress may become indeterminate;
- output FPS should use a validated fallback policy.

The system must not blindly use zero FPS.

25.7 Frame-read failure

For isolated failures:

- WARNING;
- retry if appropriate.

For consecutive failures:

- bounded retry;
- aggregate logging;
- terminate after threshold.

25.8 Model missing

Startup failure.

The GUI should report:

Required YOLO model is unavailable.

The log records the actual path and diagnostic cause.

25.9 Corrupt model

Startup failure.

Do not retry repeatedly.

25.10 Invalid model configuration

Startup failure.

Examples:

- invalid model path;
- incompatible configuration;
- invalid device;
- incompatible tracker configuration.

25.11 CUDA initialization failure

If:

DEVICE = cuda

and CUDA cannot be initialized:

- ERROR;
- abort startup;
- do not silently switch to CPU.

If:

DEVICE = auto

CPU fallback may be allowed.

25.12 Inference exception

An isolated inference failure should be handled according to configured policy.

The application may retry a limited number of times.

If the inference failure threshold is exceeded:

- ERROR;
- stop processing;
- close writer;
- release reader;
- clean temporary output as appropriate;
- return GUI to failed state.

Continuing to write a video without reliable detection results would compromise output correctness.

25.13 Invalid detection output

Malformed detector results shall be rejected.

The application shall log the problem and prevent malformed coordinates or confidence values from reaching the renderer.

25.14 Tracker initialization failure

Startup/processing failure.

The application shall not silently fall back to detector-only operation because tracking is a functional requirement.

25.15 Tracker exception

An isolated exception may be handled according to a bounded policy.

A persistent tracker failure terminates processing.

The application must not continue while pretending persistent identity is still valid.

25.16 Identity-management failure

If the target manager cannot confidently associate a new track with an existing target:

- preserve the original target reservation;
- do not assign another dog to that target;
- log the identity ambiguity;
- continue tracking other targets where possible.

This is preferable to silently changing target identity.

25.17 Output creation failure

If temporary output cannot be created:

- ERROR;
- stop processing;
- release input;
- report failure.

25.18 Insufficient disk space

The writer may fail during processing.

The application should:

- stop processing;
- close resources;
- retain useful diagnostic information;
- report that output storage is unavailable.

25.19 Output write failure

A failed frame write invalidates the resulting video.

Processing should terminate rather than silently producing an incomplete output.

25.20 Playback failure

If the temporary video exists but cannot be played:

- log ERROR;
- retain the file where possible;
- keep Save available if the output itself is valid;

- report playback failure separately from processing failure.

25.21 GUI initialization failure

The application should terminate cleanly.

No processing should begin without a functional GUI.

25.22 User cancellation

Cancellation should:

1. signal the processing worker;
2. stop after a safe processing boundary;
3. close writer;
4. release reader;
5. clean temporary output unless the design explicitly preserves partial output;
6. update GUI state.

V1 should treat cancelled output as invalid and delete it.

25.23 Unexpected exception

The controller shall have a top-level exception boundary.

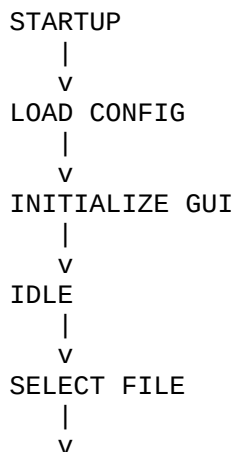
The log should contain the diagnostic traceback.

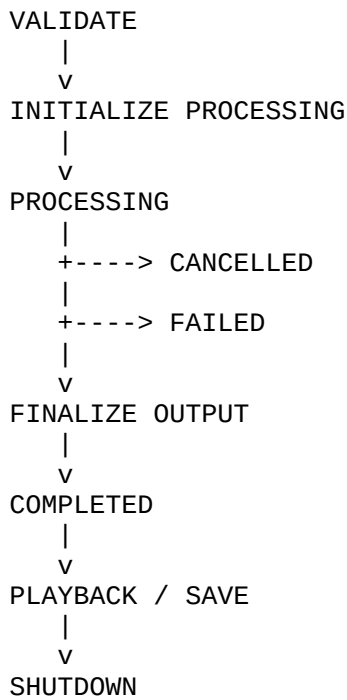
The GUI should show a generic message:

An unexpected error stopped processing.
See the application log for details.

26. Application Lifecycle and Cleanup

The lifecycle is:





Cleanup must be idempotent.

The system should safely handle:

normal completion
user cancellation
processing failure
unexpected exception
window close
save failure

Resources requiring cleanup include:

- VideoCapture;
- VideoWriter;
- temporary files;
- worker thread;
- GUI resources;
- playback resources;
- model/runtime resources where applicable.

There is no camera resource in V1.

27. Testing and QA Architecture

All automated tests shall reside under:

test/

Tests should be divided by responsibility.

Example:

```
test/  
  unit/  
  integration/  
  gui/  
  performance/  
  e2e/
```

Unit tests

Test:

- configuration;
- detection conversion;
- dog filtering;
- target selection;
- duplicate prevention;
- target lifecycle;
- association logic;
- performance calculations;
- logging behavior.

Integration tests

Test combinations such as:

`mock detector + real target selector + mock tracker`

and:

`mock detector + real tracker adapter + real target manager`

where practical.

Error-path tests

Explicitly inject:

- failed video opening;
- failed frame reads;
- corrupt frames;
- detector exceptions;
- tracker exceptions;

- writer failures;
- insufficient output storage simulation;
- invalid model configuration.

GUI tests

Test:

- file selection;
- button enable/disable state;
- progress state;
- error display;
- completed state;
- Save flow.

Full video playback automation may be limited because GUI/video backend behavior is platform-dependent.

End-to-end tests

Use a small deterministic test video.

The test should verify:

- model loads;
- dogs are detected;
- first N are selected;
- identities remain consistent;
- annotations are produced;
- output can be opened.

Real model predictions belong primarily in integration/end-to-end tests, not unit tests.

28. Mock Components and Test Isolation

The following components should be mockable:

VideoReader
Detector
Tracker
Renderer
VideoWriter
PerformanceMonitor
SaveManager
GUI event interface

A fake detector can return predetermined detections:

Frame 1:
dog A

Frame 2:
dog A

Frame 3:
dog A + dog B

This permits deterministic validation of first-N behavior without running YOLO.

A fake tracker can produce:

Frame 1: tracker 7
Frame 2: tracker 7
Frame 3: tracker 7

and later:

Frame 20: tracker 12

to test target re-association logic.

This is especially important because tracker behavior is difficult to test reliably using arbitrary real video.

29. Performance Testing

Performance testing should use representative prerecorded videos.

At minimum:

Test A: CPU

Small YOLO model, CPU-only.

Measure:

- processing FPS;
- inference time;
- memory;
- CPU.

Test B: GPU

Same video and model on supported CUDA hardware.

Measure the same metrics.

Test C: High-resolution input

Tests decoder, preprocessing, inference, and output-writing overhead.

Test D: Multiple dogs

Use videos with:

- one dog;
- several dogs;
- similar-looking dogs;
- crossing dogs;
- occlusions;
- dogs entering/leaving frame.

Test E: Long video

Evaluate:

- memory stability;
- tracker stability;
- temporary output growth;
- performance degradation;
- logging behavior.

The system should be evaluated for resource leaks by comparing memory and processing behavior across long workloads.

30. Security and Local Data Handling

The application has no network component.

Therefore:

- no video leaves the machine;
- no detection data is transmitted;
- no cloud inference occurs;
- no telemetry is required.

Temporary files should be created in an application-managed temporary location.

Temporary filenames should not be predictable enough to create unnecessary collision risk.

The application should avoid writing sensitive video content into log files.

Logs should contain:

- paths when useful for diagnostics;
- processing status;

- technical errors.

Logs should not contain:

- frame pixel data;
- complete video content;
- unnecessary copied detection histories.

Permanent output is created only after explicit Save.

31. Scalability and Future Version Strategy

Scalability is achieved through boundaries rather than infrastructure.

31.1 Additional object categories

Current:

```
TARGET_CLASS = dog
```

Future:

```
TARGET_CLASSES = [dog, cat, person]
```

The detector already produces generic `Detection` objects, so the target-selection architecture can evolve without changing YOLO integration.

31.2 User-selectable categories

The GUI can eventually expose category selection.

V1 deliberately keeps this in configuration.

31.3 Different YOLO models

The detector interface allows:

```
UltralyticsDetector
```

to be replaced by another detector implementation.

31.4 Custom-trained models

A future configuration could specify another model path.

The application-level `Detection` representation remains unchanged.

31.5 Different trackers

The tracker interface supports:

ByteTrack
BoT-SORT
FutureTracker

without changing target management or rendering.

31.6 Additional annotations

Future versions can add:

- target IDs;
- trajectories;
- timestamps;
- speed;
- confidence history.

The renderer remains the appropriate extension boundary.

31.7 Alternative inference runtimes

Future implementations may introduce:

- ONNX;
- TensorRT;
- OpenVINO;
- other optimized runtimes.

The detector abstraction isolates this change.

31.8 Batch processing

Batch processing is deliberately not implemented in V1.

The application controller could later support:

InputJob
ProcessingJob
OutputJob

without turning V1 into a job-processing platform.

31.9 GPU optimization

Future versions can optimize:

- image size;
- batching where appropriate;
- model export;

- inference runtime;
- memory transfer.

The current architecture already isolates the inference boundary.

32. Project/File Structure

A practical structure is:

```
project/
├── src/
│   ├── main.py
│   ├── application.py
│   ├── gui/
│   │   ├── __init__.py
│   │   ├── main_window.py
│   │   ├── playback.py
│   │   └── state.py
│   ├── video/
│   │   ├── __init__.py
│   │   ├── reader.py
│   │   ├── writer.py
│   │   └── metadata.py
│   ├── detection/
│   │   ├── __init__.py
│   │   ├── detector.py
│   │   ├── ultralytics_detector.py
│   │   └── models.py
│   ├── tracking/
│   │   ├── __init__.py
│   │   ├── tracker.py
│   │   ├── ultralytics_tracker.py
│   │   └── target_manager.py
│   ├── selection/
│   │   ├── __init__.py
│   │   └── target_selector.py
│   ├── rendering/
│   │   ├── __init__.py
│   │   └── renderer.py
│   ├── processing/
│   │   ├── __init__.py
│   │   └── pipeline.py
│   ├── output/
│   │   ├── __init__.py
│   │   ├── temporary_output.py
│   │   └── save_manager.py
│   └── monitoring/
│       ├── __init__.py
│       └── performance.py
```

```
|
|  └─ logging/
|      └─ __init__.py
|          └─ logger.py
|
|  └─ config.py
|
├─ test/
|   └─ unit/
|       └─ integration/
|           └─ gui/
|               └─ performance/
|                   └─ e2e/
|
├─ models/
|   └─ .gitkeep
|
├─ test_data/
|   └─ .gitkeep
|
├─ requirements.txt
├─ README.md
└─ .gitignore
```

The exact organization may be simplified if the implementation remains small.

The purpose is to maintain clear ownership boundaries, not to maximize directory count.

33. Technology/Dependency Selection

Python

Primary application language.

OpenCV

Used for:

- video input;
- video output;
- frame operations;
- optional rendering primitives;
- playback support where appropriate.

NumPy

Used for:

- frame arrays;
- numerical operations;

- coordinate manipulation.

Ultralytics

Used for:

- YOLO model loading;
- object detection;
- tracker integration where selected.

Current Ultralytics documentation supports Python model loading and tracking, and its tracking API currently exposes ByteTrack, BoT-SORT and additional tracker implementations. ([Ultralytics Docs](#))

The exact version shall be pinned in the V1 dependency configuration after implementation validation.

PySide6

Used for:

- desktop GUI;
- file dialogs;
- controls;
- progress display;
- playback UI;
- worker/thread communication.

PySide6 is the official Qt 6 Python binding. ([Qt Documentation](#))

psutil

Recommended for:

- CPU utilization;
- memory measurements.

GPU monitoring

GPU monitoring should use the least intrusive mechanism appropriate to the selected runtime and deployment environment.

GPU metrics shall be optional.

If GPU monitoring cannot be obtained reliably, processing must continue.

PyTorch

Ultralytics uses the appropriate underlying runtime.

The application should not introduce PyTorch as an independent architectural layer merely to expose framework internals.

Rejected technologies

Web framework

Rejected because the application is a local desktop application.

Database

Rejected because V1 does not require persistent detection history.

Message broker

Rejected because the processing pipeline is local and sequential.

Distributed processing

Rejected because there is no V1 workload requiring it.

Complex DI framework

Rejected because simple constructors/protocols are sufficient.

Cloud storage

Rejected because runtime operation is explicitly offline.

34. Architectural Risks and Mitigations

Risk 1: ID switches between similar dogs

Impact: High.

A tracker can accidentally associate Dog A's detection with Dog B.

Mitigation:

- use BoT-SORT;
- optionally enable ReID;
- tune association thresholds;
- use target-level identity management;
- test crossing and occlusion scenarios;
- never treat tracker IDs as permanent application identity.

Risk 2: Same dog counted twice

Impact: High.

A tracker may create a new ID after temporary loss.

Mitigation:

- retain target reservation during tracker buffer;
- perform target-level re-association;
- use appearance information where necessary;
- do not immediately create a new target from every new tracker ID.

Risk 3: First-N slots consumed by false positives

Impact: High.

Mitigation:

- detector confidence threshold;
- candidate confirmation period;
- stable tracker association;
- deterministic candidate ordering.

Risk 4: Long disappearance

Impact: High.

No tracker can guarantee identity after arbitrary disappearance.

Mitigation:

- track buffer;
- optional ReID;
- explicit target lifecycle;
- never silently substitute another dog.

Risk 5: CPU inference too slow

Impact: Medium/High.

Mitigation:

- benchmark smaller model;
- configurable inference size;
- CPU-friendly tracker;
- optional GPU;

- performance monitoring.

Risk 6: Output codec unavailable

Impact: Medium.

Mitigation:

- validate writer initialization before processing;
- use a known supported codec;
- provide clear error;
- keep output codec configurable.

Risk 7: Temporary storage exhaustion

Impact: High for long videos.

Mitigation:

- check available storage where practical;
- monitor writer failures;
- terminate cleanly;
- delete invalid temporary output.

Risk 8: GUI becomes unresponsive

Impact: Medium.

Mitigation:

Run processing outside the GUI event loop.

A single worker thread is sufficient for V1.

This is not intended to become a multithreaded processing architecture. The worker performs processing; the GUI remains responsible for presentation and user interaction.

Risk 9: Ultralytics API changes

Impact: Medium.

Mitigation:

- isolate Ultralytics code;
- pin tested package version;
- convert results at the adapter boundary;
- maintain integration tests.

Risk 10: Unreliable video metadata

Impact: Medium.

Mitigation:

- treat frame count/FPS as optional;
 - derive progress only when reliable;
 - process until actual EOF.
-

35. Key Design Decisions and Rationale

Decision 1: Use a local desktop GUI

Selected: PySide6.

Why: Strong desktop UI facilities and a good foundation for playback and asynchronous processing.

Rejected: Web UI because it introduces a browser/server architecture that provides no V1 benefit.

Decision 2: Use OpenCV for video I/O

Selected: OpenCV.

Why: Established Python computer-vision dependency and sufficient local video functionality.

Rejected: Dedicated media-processing framework because V1 does not require its additional complexity.

Decision 3: Use Ultralytics as the YOLO implementation

Selected: Ultralytics.

Why: Provides an established Python interface and integrated object-tracking support. ([Ultralytics Docs](#))

Rejected: Building a detector runtime abstraction directly around raw PyTorch because that would unnecessarily couple V1 to model internals.

Decision 4: Wrap Ultralytics tracker APIs

Selected: Application-owned tracker abstraction.

Why: Prevents framework-specific APIs from becoming application contracts.

Rejected: Calling `model.track()` throughout the processing code.

Decision 5: BoT-SORT as primary tracker

Selected: BoT-SORT.

Why: The application prioritizes persistent identity of a fixed set of dogs, making stronger association mechanisms useful.

Rejected as primary: ByteTrack because its lower complexity and performance are attractive, but its lack of appearance/ReID information makes difficult dog-to-dog identity cases less robust.

ByteTrack remains supported as an alternative.

Decision 6: Application target IDs separate from tracker IDs

Selected: Separate identities.

Why: Tracker IDs are implementation-level temporal association identifiers and may change. The application's first-N targets need longer-lived semantics.

Decision 7: Confirmation before target assignment

Selected: Candidate confirmation.

Why: Prevents transient false positives from consuming permanent target slots.

Decision 8: Temporary processed video

Selected: Temporary file plus explicit Save.

Why: Supports playback while respecting the requirement that permanent output requires user action.

Decision 9: Single processing worker

Selected: One background worker.

Why: Keeps the GUI responsive without introducing a complex concurrency architecture.

Decision 10: Console logging

Selected: Console.

Why: Sufficient for V1 debugging and avoids unnecessary persistent logging infrastructure.

36. V1 Implementation Boundaries

The following boundaries are mandatory.

GUI boundary

The GUI knows about application states and commands.

It does not know about YOLO internals.

Application boundary

The controller coordinates processing.

It does not implement detection algorithms.

Detection boundary

The detector converts model output to `Detection`.

It does not select targets.

Selection boundary

The selector decides which candidate tracks become first-N targets.

It does not perform visual tracking.

Tracking boundary

The tracker performs temporal association.

It does not decide which targets the application cares about.

Target-state boundary

The target manager maintains application-level identities.

It does not perform neural inference.

Rendering boundary

The renderer draws annotations.

It does not determine identity.

Video boundary

The reader/writer manage video resources.

They do not understand dogs or trackers.

Output boundary

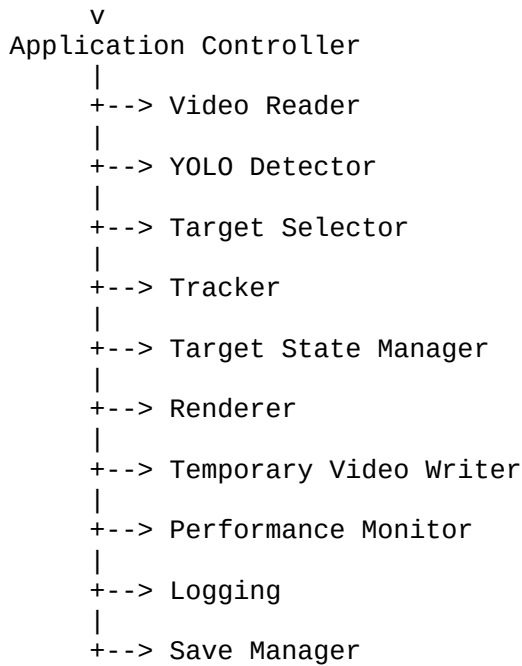
The Save Manager determines when a temporary result becomes a user-selected permanent output.

37. Summary

V1 should be implemented as a local, sequential Python desktop application with a relatively small number of clearly defined components:

PySide6 GUI

|



The most important architectural distinction is between detection, tracking, and target identity.

YOLO answers:

What objects are visible in this frame?

The tracker answers:

Which current detection corresponds to which previously observed track?

The target selector answers:

Which qualifying dogs should become the application's permanent first-N targets?

The target state manager answers:

Which application target does this current track represent?

This distinction prevents the common architectural error of treating a YOLO detection or tracker ID as a permanent object identity.

For V1, BoT-SORT is the recommended primary tracking strategy because maintaining the identity of a fixed set of dogs is more important than minimizing tracker complexity. ByteTrack remains available behind the same tracker interface for lower-overhead deployments. Current Ultralytics tracking documentation supports both and allows tracker configuration to be changed without redesigning the surrounding processing pipeline. ([Ultralytics Docs](#))

The application should use a current stable Ultralytics YOLO model selected and documented at implementation time rather than embedding an obsolete model choice into the architecture. The exact model should be chosen through accuracy/performance benchmarking on representative hardware. Current Ultralytics documentation identifies YOLO26 and YOLO11 as supported model families and provides Python APIs for detection and tracking. ([Ultralytics Docs](#))

The final output remains temporary until the user explicitly saves it. No database, network service, cloud infrastructure, persistent detection store, or distributed processing system is required.

The architecture is therefore scalable in the appropriate V1 sense: detector, tracker, target management, video I/O, GUI, and rendering have explicit boundaries that can evolve independently. It does not attempt to make V1 scalable by introducing infrastructure that the current product does not need.